MyScoutee
OFFICIAL PRODUCT DOCUMENTATION

OPERATIONS DOCUMENTATION

MyScoutee Operations Manual

Installation, configuration, availability, backup, recovery, TLS, updates, and incident response.

DOCUMENT ID	DOCUMENT VERSION
MSC-OM-001	1.4.1
APPLICABLE SOFTWARE VERSION	**STATUS**
MyScoutee 1.4.1	Released
RELEASE DATE	**LANGUAGE**
2026-10-17	EN

Document Control

Document status and applicability

Document version	1.4.1
Documentation release	2026-10-17
Document owner	MyScoutee project — operations documentation
Intended audience	System operators, infrastructure owners, security officers, release managers, and incident responders
Normative deployment method	The production system installed from the `myscoutee_1.4.1_amd64.deb` package and managed by systemd
Review triggers	Changes to the software release, deployment topology, security boundary, backup format, or operating procedure

Version History

Version	Date	Status	Change
1.4.1	2026-10-17	Released documentation	Refresh the signed 1.4.1 installer, nine-service topology, STUN/MCP/payment boundaries, retained-data migrations and upgrade verification.
1.0.0	2026-07-30	Released	Initial release for operating the MyScoutee 1.0.0 Debian package.

Version Boundary

This document describes the repository sources and operating procedures for MyScoutee 1.4.1. It may be applied to another software version only after a release-specific review has verified all differences in the package, image manifest, Compose files, and operating scripts.

Evidence Base

The procedures in this manual are derived primarily from the following version-controlled sources:

- `packaging/README.md` and `README.md`;
- `packaging/compose/docker-compose.production.yml` and the replica-set extension;
- installation and removal scripts under `packaging/debian/`;
- startup, backup, restore, TLS, update, and diagnostic utilities under `packaging/scripts/`;
- unit, path, and timer files under `packaging/systemd/`;
- `server/docker/conf/nginx/` and the production Nginx entrypoint;
- `server/projects/profile/src/main/resources/application-prod.yaml`;
- `guides/plans/PROJECT_CONTEXT.md` and `guides/plans/PRODUCTION_QUALIFICATION.md`.

Table of Contents

Purpose and Scope

Roles and Responsibilities

Production Architecture and Data Flow

Requirements and Preparation

Deployment Platforms, Public DNS, and Firebase Preparation

Initial Installation and Commissioning

Configuration and Secret Management

Service Lifecycle

Health and Operational Verification

Logging, Monitoring, and Capacity

Backup

Restore and Disaster Recovery

TLS and Certificate Lifecycle

Operator-Approved Updates

Registry Integration and Deployment Identity

Security Operations

Troubleshooting and Incident Response

Maintenance Checklists

Removal and Reinstallation

Known Operational Limitations

Command and Path Quick Reference

Appendix A — Source Build, Package Build, and Development Profile

1. Purpose and Scope

This manual provides repeatable, verifiable, and security-conscious procedures for installing and operating a MyScoutee 1.4.1 production instance.

In Scope

- Debian/Ubuntu-style amd64 hosts that use `apt`;
- the packaged stack running on Docker Engine and Docker Compose v2;
- Nginx, the Java API, MongoDB, Redis, MinIO, and the isolated Explore registry;
- startup, shutdown, health checks, log analysis, and capacity monitoring;
- backup and restore of MongoDB, configuration, MinIO, and the deployment registry;
- operation of TLS certificates, signed package updates, and registry connectivity;
- security, troubleshooting, and removal procedures.

Out of Scope

- installation and SQLite recovery of the separate Go registry service;
- end-user or administrator use of application features;
- provider-specific high-availability designs, managed databases, layer-7 load balancers, CDNs, or external backup products beyond the qualified single-VM mappings in Chapter 5;
- non-amd64 architectures, Kubernetes, or multi-node MongoDB topologies;
- guaranteed SLA, RTO, RPO, or hardware sizing.

Source Stack and Packaged Stack

The repository file `server/docker-compose-prod.yml` defines a buildable source-production environment with relative development data paths and `pull_policy: build`. The installed package deploys `packaging/compose/docker-compose.production.yml` as `/opt/myscoutee/server/docker-compose-prod.yml`, with fixed host directories and `pull_policy: never`. When operating production, use the commands under `/opt/myscoutee` specified in this manual. The two Compose variants are not interchangeable.

2. Roles and Responsibilities

Role	Primary responsibility	Non-delegable control
Deployment owner	Business approval; definition of the domain, external providers, and recovery objectives.	Ownership of Operator access and the registry identity.
System operator	Host, Docker, systemd, backup, monitoring, TLS, capacity, and incident response.	Protection of root-level secrets, the update key, and backups.

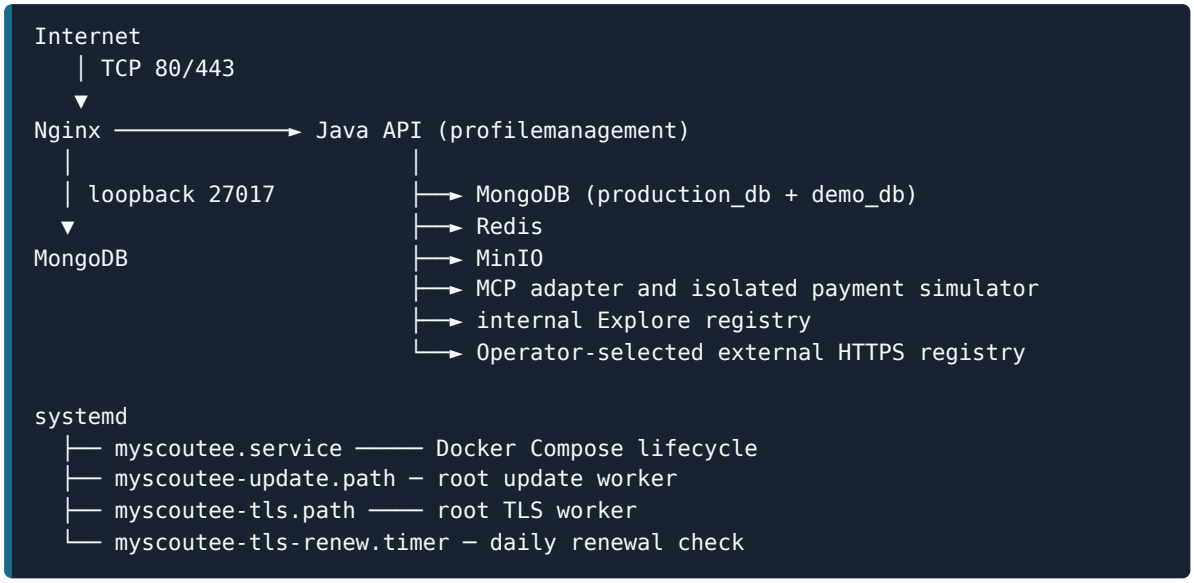
Role	Primary responsibility	Non-delegable control
Authenticated Operator	Deployment configuration, Firebase, administrators, the TLS workflow, registry catalog selection, and update approval.	Human review of the registry endpoint and scope selection and of the update manifest.
Security officer	Key and secret management, access reviews, vulnerability management, and the incident process.	Independent verification of the package-signing key fingerprint.
Release manager	Maintenance of the immutable package, image IDs, compatibility evidence, and qualification evidence.	Completion of release-specific external upgrade, restore, and security drills.

Local Decision Required

The repository does not define SLA, RTO, RPO, data-retention, or on-call targets. The deployment owner and operator must formally establish these values; backup frequency and monitoring alerts must be derived from them.

3. Production Architecture and Data Flow

Logical view of the installed production stack



Component	Container	Host access	Persistence
Reverse proxy and static frontend	`nginx`	`0.0.0.0:80`, `0.0.0.0:443`; Mongo proxy only on `127.0.0.1:27017`	TLS files on host mounts
Java backend	`myscoutee`	No direct host port; proxied by Nginx	MongoDB, registry, update, and TLS state on host mounts
MongoDB 8.0.4	`mongodb`	Via the Nginx stream proxy on loopback	`/var/lib/myscoutee/data/mongodb/data-v8-0`
Redis 8.6.1 Alpine	`redis`	Internal Compose network only	No declared volume; restartable as a cache
MinIO	`minio`	Internal Compose network only	`/var/lib/myscoutee/data/minio`
Explore demo registry	`registry-demo`	No host port	Two isolated Docker named volumes
MCP adapter	`mcp`	HTTPS through Nginx; private tools require OAuth	Authority and grants belong to the backend
Explore payment simulator	`payment-simulator`	Isolated demo provider routes	Dedicated simulator volume and credentials
Direct-call STUN	`stun`	Configured UDP/TCP STUN endpoint	Package-managed configuration; no TURN media relay

Separation of Explore and the Real Registry

The Explore registry is an internal demo service with separate SQLite data, a separate signing key, and separate Java client state. It cannot be selected as the Operator's real registry and shares no state with the external registry.

4. Requirements and Preparation

Supported Installation Platform

- An `apt`-based Debian/Ubuntu system with systemd service management;
- amd64 architecture for the released 1.4.1 package;
- root or `sudo` privileges;
- Docker Engine and Docker Compose v2;
- a public domain and DNS when Internet-facing TLS is required;
- inbound TCP 80 and 443, plus the configured UDP/TCP STUN port (3478 by default) for direct calls, while preserving a source-restricted SSH path through both the guest and provider network controls;
- outbound access at minimum for missing MongoDB/Redis images, certificate issuance, and approved external providers.

System Requirements and Initial Sizing

The following values are conservative planning baselines for the packaged nine-container stack. They are not measured capacity limits, user-count guarantees, or an SLA. Use the small-deployment column only for evaluation, commissioning, or a low-traffic deployment; use the production column as the initial production size and validate it with representative load, data-retention, media, backup, and restore tests.

Approximate host sizing baseline

Resource	Small deployment baseline	Recommended production starting point	Qualification note
CPU	4 amd64 vCPUs	8 amd64 vCPUs	Increase for concurrent API work, scheduled jobs, imports, backup compression, or restore windows.
Memory	8 GiB RAM	16 GiB RAM	Keep at least 25% headroom under representative peak load. Swap is an emergency buffer, not a replacement for RAM.

Resource	Small deployment baseline	Recommended production starting point	Qualification note
Persistent SSD storage	80 GiB	160 GiB or more	Includes the OS, Docker images, MongoDB, MinIO media, logs, update staging/cache, and local backup workspace. Retention and uploaded media normally determine growth.
Free working headroom to maintain	20 GiB	40 GiB or 25% of the filesystem, whichever is larger	Preserve additional working space before backup, restore, or update operations; do not plan to run a production filesystem close to full.
Backup capacity	Separate off-host target for at least one complete backup	Encrypted off-host target sized for the approved retention window and restore testing	Calculate from actual MongoDB, MinIO, configuration, registry, and key material. Infrastructure snapshots remain additional controls only.
Network	Stable address, canonical DNS, and connectivity for the approved providers	Stable public address and bandwidth sized from measured browser, API, media, backup, and update traffic	Latency, egress policy, upload size, and backup-window requirements matter more than the nominal link rate.

Baseline, Not a Guaranteed Minimum

The package sets no Java, MongoDB, Redis, or container memory limits, so a workload can exceed these values. Monitor CPU saturation, memory pressure, disk latency, filesystem usage, MongoDB growth, and backup duration from commissioning onward. Scale before sustained peak utilisation removes the required headroom. The update process also holds separate staging and root-cache copies and, by default, requires an additional 512 MiB free-space reserve for each copy beyond the signed artifact size.

Preinstallation Checklist

- [ ] The supported host architecture and distribution are documented.
- [ ] Time synchronization, DNS, and the hostname work correctly.
- [ ] Disk allocation separately accounts for `/var/lib`, `/var/cache`, `/var/log`, `/var/backups`, and Docker storage.
- [ ] Ports 80 and 443 are reachable through the firewall; port 27017 is not public.
- [ ] The deployment domain and TLS mode have been approved.

- [ ] A secure method is ready for recording the shown-once Operator credential.
- [ ] Owners have been assigned for the backup target, off-host copy, retention, encryption, and restore testing.
- [ ] The installation package source and SHA-256 digest have been verified.

Host Prerequisites and Optional Utilities

On a clean VM, install and test OpenSSH separately as described in 5.2. Install the product with `apt`; APT resolves the Docker Engine and Docker Compose dependencies declared by the Debian package. The following package-owned helpers become available only after package installation or explicit package extraction:

```
sudo /opt/myscoutee/packaging/scripts/install-vps-prerequisites.sh
sudo /opt/myscoutee/packaging/scripts/doctor-production.sh
```

`install-vps-prerequisites.sh` is an optional missing-utility helper, not the MyScoutee installer. `doctor-production.sh` checks only for the presence of commands; it does not verify the running stack, disk capacity, certificates, or backups.

5. Deployment Platforms, Public DNS, and Firebase Preparation

This chapter maps the supported MyScoutee host contract to common infrastructure products. Provider consoles create the VM, network, stable address, firewall policy, storage, and DNS records; they do not replace the MyScoutee package installer. The same released Debian package and the same operating procedures are used on every platform.

5.1. Runtime Boundary and Available Deployment Surfaces

The supported runtime is one amd64 Debian/Ubuntu guest with `apt`, systemd, Docker Engine, and Docker Compose v2. The package runs a stateful nine-container stack, uses fixed host paths, and installs privileged local update and TLS workers. The host does not need Java because the Java runtime is contained in the packaged API image.

Infrastructure products that can provide the supported VM

Infrastructure surface	Use for MyScoutee	Provider resources
Google Cloud	Compute Engine amd64 VM	VPC subnet, firewall rules, reserved external IP, persistent disk, and Cloud DNS or external DNS
VMware vSphere	Debian/Ubuntu 64-bit guest managed through vCenter or the vSphere Client	Datastore-backed virtual disks, port group/VLAN, stable guest address, and upstream firewall, NAT, and DNS
OpenStack	Nova amd64 instance managed through Horizon or the OpenStack API/CLI	Glance image, tested flavor, Neutron network/security group/floating IP, Cinder storage, and Designate or external DNS

Firebase and Serverless Products Are Not the Runtime

Firebase supplies external authentication and push messaging for MyScoutee; it does not host this release. Firebase Hosting/App Hosting, Google App Engine, Cloud Run, GKE/Kubernetes, and generic shared web hosting are not drop-in targets for the Debian package because they do not provide the required single-host systemd, Docker Compose, persistent-path, and privileged-worker contract.

5.2. Common VM, Storage, and Network Contract

- [ ] Select an approved amd64 Debian/Ubuntu image; do not select an Arm machine type or image.
- [ ] Keep the VM clock synchronized and give the guest a stable address that survives an ordinary stop/start.
- [ ] Begin with the Chapter 4 sizing baseline, then qualify CPU, RAM, disk performance, and capacity with release-specific load, retention, backup, and restore tests.

- [ ] Provide persistent capacity for `/var/lib`, `/var/cache`, `/var/log`, `/var/backups`, and Docker storage before installation.
- [ ] Encrypt infrastructure disks according to local policy, but retain MyScoutee logical backups and an encrypted off-host copy; a VM or volume snapshot is not a substitute for the application backup.
- [ ] Preserve administrative access without exposing database, cache, object-store, API, or registry container ports.

Minimum network-policy intent for a directly exposed VM

Traffic	Source or destination	Policy	Purpose
Inbound TCP 80	Internet	Allow	HTTP-to-HTTPS redirect, ACME HTTP-01, and the Operator domain test
Inbound TCP 443	Internet	Allow	Application HTTPS and secure WebSockets
Inbound TCP 22	Approved administrator CIDRs, bastion, VPN, or provider access proxy only	Restrict	Host administration and optional SSH forwarding
Inbound TCP 27017	Any external network	Deny	The package exposes the MongoDB proxy on host loopback only
Other container ports	Any external network	Deny	Java, Redis, MinIO, MongoDB, and the Explore registry communicate on the private Compose network
Outbound DNS, NTP, and HTTPS	Approved resolvers, time sources, image sources, ACME, Firebase/Google APIs, registry, update source, and configured providers	Allow as required	Name resolution, time, installation, certificates, authentication, messaging, and integrations

SSH Is Infrastructure Prerequisite, Not Product Installation

The MyScoutee package does not install or configure `openssh-server`, create a host-administrator account, add an SSH key, or open TCP 22. Prepare and test that management path through the provider console before relying on remote installation. On an approved Debian/Ubuntu guest, a typical host-side check is:

```
sudo apt install openssh-server
sudo sshd -t
sudo systemctl enable --now ssh
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Use a named non-root account with the required `sudo` authority and prefer key authentication. Obtain the SSH host-key fingerprint through the provider console or another independent channel, compare it on first connection, and keep console recovery available while changing SSH or firewall policy. Do not disable password login or root login until a second, key-authenticated session has succeeded.

The repository-qualified topology terminates TLS in the packaged Nginx container on the VM. A CDN or layer-7 load balancer that terminates TLS, changes redirects, or rewrites forwarded headers is a different design and requires separate qualification, including WebSocket, HSTS, health-check, client-address, and ACME behavior. A direct address, NAT, floating IP, or layer-4 pass-through is closer to the documented boundary.

5.3. Google Cloud Compute Engine Example

1. Create or select the production Google Cloud project, region, zone, VPC, and subnet under the organization’s billing, IAM, audit, and retention controls.
2. Reserve a regional static external IPv4 address. Do not publish DNS against an ephemeral address.
3. Create a firewall target for the MyScoutee VM. Allow TCP 80 and 443 from the intended public sources, restrict SSH to the approved administration path, and add no public rule for TCP 27017 or internal container ports.
4. Create a Compute Engine VM from an approved amd64 Debian/Ubuntu image. Map the Chapter 4 baseline to a machine type with at least 4 vCPUs, 8 GiB RAM, and an 80 GiB persistent SSD for a small deployment, or begin production qualification at 8 vCPUs, 16 GiB RAM, and 160 GiB. Increase storage for measured database, media, log, update, and backup retention.
5. Attach the reserved address and use OS Login, IAP, a bastion, or approved SSH keys. The VM’s Google Cloud service account needs no broad Firebase role; grant only permissions required for separately selected Google Cloud agents or backup automation.
6. Create the canonical DNS record described in 5.6, wait for public propagation, and verify that it resolves to the reserved address. Verify host-to-public-name reachability after the package and Nginx have started.
7. Transfer the approved `myscoutee_1.4.1_amd64.deb`, verify its SHA-256 digest, and continue with Chapter 6. Take the first logical backup after commissioning; Compute Engine disk snapshots remain an additional infrastructure control only.

Do Not Confuse Google Identities

The human or automation identity that provisions Google Cloud, the service account optionally attached to the Compute Engine VM, and the JSON service-account credentials supplied to MyScoutee for Firebase are separate security principals. An attached VM service account does not replace the Firebase credentials expected by the current application interface.

5.4. VMware vSphere Mapping

1. In vCenter or the vSphere Client, create a Debian/Ubuntu 64-bit VM from an approved template or ISO and attach it to the intended datastore and port group/VLAN.
2. Assign vCPU, RAM, and virtual-disk capacity from the Chapter 4 baseline, then adjust it from load-test and retention evidence. Install the approved guest tools package when required by the local vSphere operating standard.
3. Give the guest a stable address through static guest configuration or a controlled DHCP reservation. Use the vSphere console to install and validate OpenSSH, provision the named host-administrator account and public key, and record the guest's SSH host-key fingerprint.
4. Prefer a private management port group reached through the organization's VPN or bastion. Permit TCP 22 from only that management source in the guest firewall and in any NSX distributed firewall or upstream firewall that protects the VLAN.
5. If the guest must be reached through NAT, create a narrowly scoped DNAT/VIP rule on NSX Edge or the upstream firewall/router, for example external TCP 2222 to the guest's TCP 22, and permit only the approved administrator CIDR. A vSphere port group connects the virtual NIC; vSphere alone does not create the required Internet NAT or perimeter-firewall policy. Set `REMOTE_HOST` and `REMOTE_PORT` in the client tools to the externally reachable address and port.
6. From the actual management workstation, compare the independently obtained host-key fingerprint and verify `ssh -i /path/to/key -p 2222 host-admin@198.51.100.20`. Remove temporary broad rules after this test; do not expose SSH to `0.0.0.0/0`.
7. For public application access, separately configure the upstream firewall and one-to-one NAT or virtual IP so TCP 80 and 443 reach the same guest. Point public DNS at that stable address and verify NAT reflection/hairpin behavior as described in 5.6.

Do not clone a running or previously registered production VM to create another deployment. That duplicates MongoDB data, the registry signing identity and receipts, the deployment master key, and provider credentials. Move or restore one logical deployment as a controlled recovery unit. vSphere snapshots are short-lived infrastructure rollback aids, not database-aware MyScoutee backups.

5.5. OpenStack Mapping

1. Select an approved amd64 Debian/Ubuntu Glance image and map the Chapter 4 CPU, RAM, and SSD baseline to a Nova flavor and Cinder-backed boot volume. Select the tenant network, inject an approved OpenStack key pair at server creation, and confirm the image-specific non-root login name.

2. Create a dedicated Neutron administration security group with one ingress TCP 22 rule whose remote CIDR is the administrator workstation, VPN, or bastion—not the entire Internet. Attach that group to the instance port. Create separate public-web rules for TCP 80 and 443; do not add TCP 27017 or internal container ports.
3. For a directly administered instance, allocate a floating IP from the approved external network and associate it with the server. If policy requires a private instance, do not allocate a floating IP: connect through a VPN or bastion and restrict TCP 22 to that private source or security group.
4. Equivalent OpenStack CLI operations are:

   ```
   openstack security group create myscoutee-admin
   openstack security group rule create \
     --ingress --ethertype IPv4 --protocol tcp --dst-port 22 \
     --remote-ip 203.0.113.10/32 myscoutee-admin
   openstack server add security group myscoutee-vm myscoutee-admin
   openstack floating ip create public
   openstack server add floating ip myscoutee-vm 198.51.100.42
   ```

   Replace the documentation addresses, server name, external-network name, and source CIDR with approved values. The floating-IP creation result supplies the real address used by the association command.
5. From the management workstation, verify the host key obtained through the console and connect with `ssh -i /path/to/key host-admin@198.51.100.42`. When a bastion is required, configure OpenSSH `ProxyJump` or an equivalent approved route and place the corresponding option in the client configuration.
6. Create the canonical A record in Designate when the cloud offers it, or in the organization’s external authoritative DNS. Verify the floating-IP and self-route behavior described in 5.6.
7. Install the same released Debian package in the guest and use MyScoutee backup/restore procedures in addition to any Cinder snapshot or volume-backup policy.

OpenStack deployments vary: Horizon, Cinder, or Designate may be absent. Equivalent provider APIs, persistent block storage, and external DNS are acceptable when they satisfy the same host, persistence, network, and recovery contract.

5.6. Canonical Domain, DNS Records, and NAT Self-Access

Choose one canonical application FQDN, for example `app.example.com`. The value of `DOMAIN` is the DNS name only—without `https://`, a port, a path, or a wildcard. The matching browser origin, for example `https://app.example.com`, must appear in `WEBSOCKET_ALLOWED_ORIGIN_PATTERNS`.

Public DNS records for the canonical deployment

Record	Use	Guidance
`NS`	Zone or subzone delegation	Change registrar or parent-zone delegation only when Cloud DNS, Designate, or another provider becomes authoritative.
`A app.example.com`	Canonical IPv4 endpoint	Required for the direct-VM example; target the reserved address, public NAT/VIP, or floating IP.

Record	Use	Guidance
`AAAA app.example.com`	Canonical IPv6 endpoint	Publish only after IPv6 routing, firewalling, Nginx reachability, and TLS have passed external tests. A broken AAAA record can make the application intermittently unreachable.
`CAA`	Permitted certificate authorities	Optional; if the zone uses CAA, authorize Let’s Encrypt or the operator-selected CA before issuance.
`CNAME`	Alias or provider front-door hostname	Not required for the canonical direct-VM deployment. Additional aliases and their certificates/redirects are not qualified by this single-domain package workflow.

Lower the DNS TTL before a planned cutover, then restore the organization’s normal TTL after the stable address is confirmed. Verify from an external resolver and from the VM:

```
dig +short A app.example.com
dig +short AAAA app.example.com
curl --fail --silent --show-error http://app.example.com/nginx-health
```

The Operator TLS domain test resolves the configured public name and requests its ACME challenge URL from the deployment host itself. A VMware or OpenStack VM behind NAT/VIP therefore needs working NAT reflection/hairpin routing, or a controlled split-DNS route that still reaches the same packaged Nginx endpoint. Test from an unrelated external network as well. Do not bypass a failed test by disabling certificate verification.

5.7. Firebase Project, Web App, and Server Credentials

Firebase is independent of the VM provider. Add Firebase to an approved Google Cloud project or create a dedicated production Firebase project, then register a Web app. Firebase Hosting is not required. Use the Web app configuration to collect the public client values: API key, default auth domain, project ID, storage bucket, messaging sender ID, app ID, optional measurement ID, and the public VAPID key.

1. In Firebase Authentication, enable both Email/Password and Google sign-in. Configure the Google provider’s approved support identity.
2. Add the canonical MyScoutee FQDN and the Web app configuration’s default `<project-id>.firebaseapp.com` auth domain to Authentication’s authorized domains. Keep the default Firebase auth domain in MyScoutee; a custom auth domain requires a separate Firebase Hosting/auth-handler design that this chapter does not implement.
3. Enable Firebase Cloud Messaging and the APIs required for web registration and HTTP v1 sending. Generate or import a Web Push key pair and copy only its public VAPID key into the public-client field.
4. Create dedicated service accounts in the same Firebase/Google Cloud project. A least-privilege starting point is Firebase Authentication Viewer (`roles/firebaseauth.viewer`) for the authentication credential and Firebase Cloud Messaging API Admin (`roles/firebasecloudmessaging.admin`) for the messaging credential. Confirm local policy and the built-in capability tests before granting anything broader.

5. Create JSON keys only because the current MyScoutee interface requires complete service-account JSON documents. Treat each private key like a password: never commit it, bake it into an image, put it in browser configuration, or send it through email, chat, or a ticket.
6. Sign in with the deployment-local bootstrap Operator and open **Operator → Configuration**. Enter the public values and paste the complete authentication and messaging JSON credentials into their separate fields. The two credentials and every public value must belong to the same project.
7. Select **Save Firebase**, run **Auth Test**, allow browser notifications and run **Message Test**, then select **Activate Firebase** only after both tests pass. Verify sign-in, token refresh, notification permission, and a test notification in a fresh browser session.

> ### The Browser API Key Is Not the Server Key
>
> The Firebase API key and VAPID public key are public browser bootstrap values. The server credential is the complete service-account JSON containing a private key. MyScoutee encrypts Operator-managed credentials in MongoDB with the deployment master key and never returns their plaintext. Back up and restore that database and its matching master key as one unit.

For the current Operator-managed workflow, leave the startup compatibility flag `FIREBASE_ENABLED=false`. The package creates a non-secret `{}` placeholder for its declared Compose secret, starts with the deployment-local Operator credential, and uses the separately saved and activated encrypted credentials at runtime. Do not enable the legacy file-backed startup path for a new deployment. Perform key rotation by saving new JSON credentials, repeating both tests, activating the new revision, verifying a fresh session, and only then deleting the old Google Cloud keys.

5.8. Package Boundary, Operator Client Tools, and SSH Forwarding

A release build produces two deliverables with different execution locations. Neither changes the VM provider’s network or SSH policy.

Release installation and management artifacts

Artifact or path	Execution location	Purpose
`myscoutee_1.4.1_amd64.deb`	Target Debian/Ubuntu VM	The product installer. `apt` resolves dependencies and invokes package configuration.
`myscoutee-client-tools_1.4.1.tar.gz`	Trusted Operator management workstation	Release-packaged verifier, SSH/SCP wrappers, secret-free configuration example, and external read-only deployment smoke.

Artifact or path	Execution location	Purpose
`myscoutee-release_1.4.1.json`, release public key, and independently obtained key fingerprint	Trusted Operator management workstation	Signed metadata and trust inputs used to authenticate both the Debian package and the client-tools archive.
`/usr/share/myscoutee/myscoutee-client-tools_1.4.1.tar.gz`	Target VM storage; transfer to a trusted Operator management workstation	A byte-identical, version-matched copy of the client archive. It is not expanded or executed on the deployment host.
`/opt/myscoutee/packaging/scripts/`	Target VM, normally through `sudo`	Package-owned service, qualification, diagnostics, backup, restore, TLS, and update commands.

The Debian package creates its directories and local secrets, loads the bundled application images, obtains missing pinned service images, and enables its systemd units. Direct `apt` installation deliberately leaves the application stack stopped for configuration. The client `install.sh` wrapper normally restarts and qualifies it because `START_SERVICE=true` is the client-bundle default.

The workstation bundle is GNU/Linux-oriented. It requires OpenSSH `ssh`/`scp`, APT package-inspection utilities (`dpkg` and `dpkg-deb`), OpenSSL, `jq`, `tar`, `base64`, `od`, `sha256sum`, GNU `find`/`stat`/`ps`, and the usual core utilities. Password transport additionally requires `sshpass` or `setsid`; key transport is preferred. Node.js 20 or later is recommended for the external smoke runner. The remote host must provide the supported Debian/Ubuntu APT environment, root or adequate `sudo`, and the separately prepared SSH service.

Authenticate and prepare the Operator client bundle

An authorized Operator with SFTP access can download the exact version-matched archive from an already installed host to the administrative workstation. Use the workstation's approved SSH configuration; do not place a password or private-key content in the command:

```
sftp myscoutee-host
sftp> get /usr/share/myscoutee/myscoutee-client-tools_1.4.1.tar.gz ./myscoutee-client-tools_1.4.1.tar.gz
sftp> bye
```

SFTP provides transport, not release authenticity. The archive must be authenticated before extraction, and every client helper—including `install.sh`, `forward-localhost.sh`, and `verify-deployment`—must be extracted and executed only on the trusted workstation, never on the deployment host.

Authenticate the client archive before extracting or executing anything from it. Use an already trusted copy of `verify-release.sh`—for example, one retained from a previously approved release, extracted from an independently authenticated package, or distributed through the organization’s controlled verifier channel:

```
/approved/trusted/verify-release.sh \
  myscoutee-client-tools_1.4.1.tar.gz \
  myscoutee-release_1.4.1.json \
  release-public-key.pem \
  sha256:<independently-obtained-64-lowercase-hex-characters>

mkdir myscoutee-client-tools-1.4.1
tar -xzf myscoutee-client-tools_1.4.1.tar.gz \
  -C myscoutee-client-tools-1.4.1
cd myscoutee-client-tools-1.4.1
cp install.env.example install.env
chmod 0600 install.env
```

Do not bootstrap trust by running the verifier from the same unauthenticated archive it is supposed to verify. The pinned fingerprint authenticates the release public key; it does not authenticate arbitrary verifier code obtained beside that key.

Set the SSH endpoint established in 5.3–5.5. A key-based production example is:

```
REMOTE_HOST=198.51.100.20
REMOTE_USER=host-admin
REMOTE_PORT=22
REMOTE_KEY_FILE=/absolute/path/to/id_ed25519
REMOTE_HOST_KEY_CHECKING=yes
START_SERVICE=true
STOP_ON_FAILURE=true
QUALIFY_TIMEOUT_SECONDS=300
RELEASE_MANIFEST_PATH=/approved/path/myscoutee-release_1.4.1.json
RELEASE_PUBLIC_KEY_PATH=/approved/path/release-public-key.pem
RELEASE_PUBLIC_KEY_FINGERPRINT=sha256:<64-lowercase-hex-characters>
```

The remote account must already exist, accept SSH/SCP, and be root or have the `sudo` authority required for APT, systemd, Docker, and package-owned host commands. If a bastion is required, use the workstation's OpenSSH configuration or an approved quoted `REMOTE_SSH_OPTIONS` value such as `"-J bastion-user@bastion.example.com"`. Prefer a dedicated SSH key. If `REMOTE_PASSWORD` or `REMOTE_SUDO_PASSWORD` is used, the helper rejects an `install.env` mode other than 0600 or 0400. The real `install.env` is deliberately absent from both release artifacts.

> ### Pin the Host and Verify the Package
>
> The example selects strict known-host checking. Populate the workstation's `known_hosts` only after comparing the guest fingerprint through the provider console or another independent channel. Obtain the full release-key SPKI fingerprint through an independent release-authority channel as well; a public key downloaded beside the artifacts is not its own trust anchor.

Install, reinstall, or perform an approved manual upgrade

```
./install.sh /approved/path/myscoutee_1.4.1_amd64.deb
```

Before uploading anything, `install.sh` calls the packaged `verify-release.sh`. For a published release, the verifier requires all three configured release inputs and checks the independently pinned Ed25519 SPKI fingerprint, canonical signing-key ID and signatures for both release artifacts, declared size and SHA-256, canonical filenames, and the Debian package identity and

version. It refuses an unverified package. `ALLOW_UNSIGNED_LOCAL_PACKAGE=true` is an explicit development-only escape hatch for an artifact built directly from a trusted local checkout; never use it for a downloaded or published artifact.

After verification, `install.sh` uploads the package to the configured simple path under `/tmp`, reads the installed version, calls remote `apt-get install`, and then invokes the package-provided `qualify-deployment.sh`. A same-version operation uses APT `--reinstall` without removing the package first. A downgrade is refused unless `ALLOW_DOWNGRADE=true` was explicitly approved. `RUN_APT_UPDATE`, `START_SERVICE`, `STOP_ON_FAILURE`, and the qualification timeout remain explicit local policy inputs. Setting `START_SERVICE=false` selects qualification of an already active service; it does not skip qualification or start a stopped service.

The wrapper does not run the public deployment smoke. `SMOKE_PUBLIC_BASE_URL` only makes it print the exact follow-up command; the external check must still run from the management workstation as described in 6.6.

> ### Installable and Upgradeable Does Not Mean Self-Updating
>
> A `.deb` file never upgrades itself. A newer package can be applied directly with `apt` or through the client wrapper, but both are manual administration paths. The installed product also contains the Operator-approved update path in Chapter 14: after explicit approval it verifies a newer signed `.deb`, requires a complete backup, invokes local `apt-get install`, restarts the service, and verifies the running images. It never polls and installs releases automatically, accepts only a version newer than the installed version, and performs no automatic data rollback.

Package-provided host commands

Command	Operator use
`qualify-deployment.sh`	Explicit `restart-and-check`, `check`, or `diagnostics` modes; never invoked automatically by Debian `postinst`.
`deployment-diagnostics.sh`	Read-only systemd status, MyScoutee container/Compose status, and recent service journal.
`doctor-production.sh` and `status-production.sh`	Prerequisite-command presence and current Compose status respectively.
`start-production.sh`, `stop-production.sh`, and `myscoutee-compose.sh`	Package-aware stack lifecycle and controlled direct Compose access. Prefer systemd for routine service lifecycle.
`backup-production.sh` and `restore-production.sh`	Logical deployment backup and guarded restore described in Chapters 11 and 12.
`myscoutee-update-worker.sh` and `myscoutee-tls-worker.sh`	Privileged fixed-request workers started by package-owned systemd units; not interactive general-purpose shell entry points.

Other files in the installed scripts directory support package build, validation, migration, or internal worker boundaries. Their presence does not make every script a supported routine production command.

Use the packaged SSH forward

After the same `install.env` target has been verified, the client bundle can create background, loopback-only forwards. Its defaults are:

```
sudo -E ./forward-localhost.sh
```

```
local 127.0.0.1:80  → remote 127.0.0.1:80
local 127.0.0.1:443 → remote 127.0.0.1:443
```

Local ports 80 and 443 require local privilege. To keep the local end unprivileged, override the mappings for one invocation:

```
LOCAL_FORWARD_PORTS_OVERRIDE=18080:80,18443:443 \
  ./forward-localhost.sh
```

Each comma-separated item is `local[:remote]`; one number means the same port at both ends. The script refuses a non-loopback local or remote bind, verifies that each forward starts, and replaces an exact existing SSH session for the same target and mapping. It uses background `ssh -fN` sessions and provides no stop command, so identify and terminate only the exact matching SSH process when diagnostics finish.

For a foreground one-off tunnel that ends with `Ctrl+C`, use OpenSSH directly:

```
ssh -N -o ExitOnForwardFailure=yes \
  -p 22 -i /absolute/path/to/id_ed25519 \
  -L 127.0.0.1:18080:127.0.0.1:80 \
  -L 127.0.0.1:18443:127.0.0.1:443 \
  host-admin@198.51.100.20
```

After installing a canonical-domain certificate, keep that domain as the TLS hostname and SNI while resolving only this high-port request to local loopback:

```
curl --resolve app.example.com:18443:127.0.0.1 \
  https://app.example.com:18443/api/health
```

`https://localhost:18443` validates only when the selected certificate covers `localhost`, such as the package’s bootstrap certificate. For a private CA, supply its CA file to the client; do not disable certificate verification.

A tunnel does not install MyScoutee, create a public endpoint, change provider firewall rules, or prove public DNS, ACME, redirect, WebSocket, or Internet reachability. Do not use `-k` to hide a certificate failure. Complete the independent public checks in Chapters 6 and 13.

5.9. Worked Example: Mainland-China Country-Local Deployment

This example demonstrates the platform’s country-local capability in a demanding environment; the same pattern applies to other sovereign, disconnected, or policy-restricted networks. MyScoutee can run on an approved Debian/Ubuntu VM in a mainland regional cloud, local

VMware/OpenStack environment, or authorized data centre. The application, persistent data, signed release artifacts, backups, monitoring, DNS, and—when required—a separately operated MyScoutee registry can remain on approved in-country network paths.

Country-Local Operating Outcome

Routine application operation does not require an always-available global registry. A deployment may remain registry-disabled, or it may select a qualified country-local registry so registration, signed feeds, and registry reporting stay regional. The standard signed Debian and client artifacts can be transferred through an approved offline or in-country release channel; GitHub, Docker Hub, and another foreign public service do not need to be deployment-time dependencies.

The objective is deployment locality and compliance, not circumvention of national controls. Qualify every required path from the real user and operator networks with the regional provider, security owner, and legal/compliance counsel.

Example deployment sequence

1. Provision an amd64 Debian/Ubuntu VM on an authorized mainland provider or approved local infrastructure. Start with the Chapter 4 production baseline, assign a stable address, allow public TCP 80/443, and restrict SSH to the local administration network or bastion.
2. Prepare the in-country domain, authoritative DNS, TLS route, ICP process, and any other required approvals before public launch.
3. Transfer the approved signed release set into an in-country artifact service through a controlled import path. Pre-stage the Debian dependencies and all container images so installation, restart, certificate recovery, and rollback do not depend on GitHub, Docker Hub, or another cross-border source.
4. Install and qualify MyScoutee through the same Debian/client-tools workflow used elsewhere. Keep monitoring, audit evidence, and encrypted backups on approved local services in separate failure domains.
5. Choose the registry mode: operate locally with registry traffic disabled, or select a partnership-provided mainland MyScoutee registry endpoint and scope so registry reporting and signed feeds also remain regional.
6. Test commissioning while cross-border access is deliberately unavailable. Enable only those external integrations—including Firebase—that are explicitly approved and proven reachable from both server and client networks.

Registry choices demonstrated by the country-local example

Choice	Current behavior	Operational consequence
Local application without a real registry	Keep `REGISTRY_ENABLED=false` and `REGISTRY_OUTBOUND_ALLOWED=false`. Core local application operation does not require automatic registry traffic.	Registry registration, signed feeds, network functions, and the normal Operator update-announcement path are unavailable. Before registry initialization, the complete-backup requirement also blocks the privileged update rail.

Choice	Current behavior	Operational consequence
Local application with a jurisdiction-local MyScoutee registry	Run a separately qualified registry endpoint in the same authorized network, with its own scope, database, signing key, TLS identity, monitoring, and backup. Select that endpoint through the Operator workflow in Chapter 15.	Application-to-registry traffic stays on the approved regional path. Release artifacts referenced by regional update announcements must also be reachable and independently signed.
Global and regional registries	Separate registry services may coexist for different deployment populations, but one MyScoutee deployment selects exactly one real endpoint and scope at a time.	There is no current active-active selection, forwarding, relay, cross-scope replication, automatic fallback, or transparent failover. Moving one deployment requires the signed unregister/change process in 15.4.

Localize every runtime dependency, not only the VM

- [ ] Import the authenticated Debian package, client archive, signed release manifest, and public key through an approved transfer path; authenticate the full release-key fingerprint through a separate channel.
- [ ] Mirror the exact vendor-signed package bytes to an approved in-country HTTPS artifact service rather than rebuilding them. A regional update announcement can reference that reachable copy while the pinned release signature, size, and SHA-256 continue to authenticate the artifact.
- [ ] Provide approved local APT sources and pre-stage every required container image. The standard package contains the five first-party images but may still need MongoDB and Redis images, and TLS operations need the approved Certbot image.
- [ ] Use reachable authoritative DNS, resolvers, time sources, monitoring, alerting, and an encrypted backup target in a separate local failure domain.
- [ ] Use a locally accepted public CA, a reachable approved ACME service, or the manual/private-CA TLS workflow. Do not disable certificate verification.
- [ ] Allow only explicitly approved HTTPS destinations for the selected registry, signed update artifacts, and other configured providers. Test both allowed traffic and the intended denials.
- [ ] Treat Firebase separately. A local VM and local registry do not localize Firebase Authentication or Cloud Messaging. Test those Google endpoints from the actual network before promising those features. If they are not approved and reliably reachable, keep Firebase inactive and record the feature limitation; replacing Firebase requires a separately developed and qualified product integration.

Mainland-China Implementation Path

Use an authorized mainland provider or approved local VMware/OpenStack infrastructure; Google Cloud's current Compute Engine list includes Hong Kong but does not list a mainland-China region, and Hong Kong is not a mainland-local substitute. Before a public launch, complete the access provider's ICP process and confirm whether PSB filing, a commercial ICP licence, or sector-specific approval applies. Treat provider, entity, domain, content, cybersecurity, privacy, and cross-border-data review as commissioning workstreams. This manual provides the technical deployment model; the regional provider and qualified counsel determine the applicable legal process.

Regional Registry Partnership

Where registry functions must also remain country-local, a separately governed MyScoutee registry can be supplied and qualified through a regional technical partnership. Country-local deployments select that regional HTTPS endpoint and scope, while deployments elsewhere may use a different global registry. Each application deployment binds to one real registry at a time; the application package's isolated Explore registry is never used as the real service. The MyScoutee registry is distinct from the private OCI image repository used to distribute container images inside the region. Its server release, technical operating material, and host administration form a restricted regional-partnership workstream separate from ordinary MyScoutee application operations.

Please contact MyScoutee if you want to deploy this topology. This is a partnership deployment: the real registry server is not publicly distributed in release 1.4.1, and country-local registry delivery and qualification are arranged for the target environment.

Restricted Regional Registry Delivery and Access

Confidential — controlled/internal distribution only. The separate Registry server release and its technical operating material may be provided only to expressly authorized regional technical partner personnel through the partnership's controlled distribution process. Registry host access is restricted to those personnel under the applicable access-control and audit procedures. This access is not part of ordinary MyScoutee Operator access, and neither Registry delivery nor host administration is public or self-service.

5.10. Current Provider References

- Google Cloud — create a Linux VM and Cloud DNS record management;
- Google Cloud Compute Engine region and zone list;
- Firebase Web app setup, Admin SDK credentials, web messaging/VAPID setup, FCM network requirements, and product-level IAM roles;
- Ubuntu Server OpenSSH guidance;
- Broadcom vSphere distributed port-group reference and VM network troubleshooting;

- OpenStack Horizon instance guidance, Nova instance guidance, network CLI reference, compute CLI reference, and Designate record-set guidance;
- China MIIT non-commercial filing rules and commercial-licence/non-commercial-filing distinction;
- Alibaba Cloud’s mainland-China ICP guidance for overseas enterprises and regional OCI image-repository controls as one provider example.

Provider interfaces and product names change independently of MyScoutee. Recheck the current vendor documentation and local cloud policy during every infrastructure qualification.

6. Initial Installation and Commissioning

6.1. Verify the Package

For a published release, use the authenticated client workflow in 5.8 so the signed release manifest, Ed25519 signature, pinned key fingerprint, artifact size, SHA-256, and Debian identity are all verified before upload. The following inspection is useful additional evidence, but a bare digest without an independently authenticated value is not release authentication:

```
sha256sum myscoutee_1.4.1_amd64.deb
dpkg-deb --info myscoutee_1.4.1_amd64.deb
```

Compare the SHA-256 value with the approved release evidence. Do not install a package of unknown origin or with a mismatched digest.

6.2. Install

Use direct APT for an initial production installation when the host configuration must be reviewed before the first application start:

```
sudo apt install ./myscoutee_1.4.1_amd64.deb
```

> ### Do Not Use `dpkg -i` by Itself
>
> The package declares dependencies on Docker Engine and Docker Compose v2. The documented installation path is `apt install ./…deb`, which resolves dependencies before post-installation configuration.

Alternatively, the authenticated `client-tools/install.sh` path in 5.8 uploads through SSH, runs APT, and performs package-provided host qualification. Its default `START_SERVICE=true` immediately selects `restart-and-check`, so use that path for a fresh deployment only when starting with the packaged bootstrap configuration is approved. `START_SERVICE=false` means “qualify an already running service”; it is not a fresh-install pause switch.

During the initial installation, the post-installation process:

1. creates the configuration, data, registry, update, TLS, log, and backup directories;
2. creates the Operator configuration encryption key from 32 cryptographically random bytes represented as 64 lowercase hexadecimal characters;
3. generates separate passwords for the MongoDB root user, the MongoDB application user, and MinIO;
4. when `OPERATOR_USER` is empty, creates a deployment-local Operator identifier and a random 256-bit password;
5. prints the password once, but stores only the identifier and the lowercase SHA-256 digest of the password;
6. loads the five packaged MyScoutee images, verifies the image manifest, and pulls the configured MongoDB and Redis images if they are missing;

7. enables `myscoutee.service`, the update and TLS path units, and the TLS renewal timer;
8. does not automatically start the MyScoutee application stack.

> **Shown Once Does Not Mean Single Use**
>
> Immediately store the deployment-local Operator password displayed during installation in a secure secret store, and protect package-manager, terminal, and automation logs. The password remains valid for subsequent local Operator sign-ins until the deployment owner disables or replaces it; enabling Firebase alone does not invalidate it.

6.3. Configure Before First Startup

```
sudoedit /etc/myscoutee/myscoutee.env
sudoedit /etc/myscoutee/myscoutee.secrets.env
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh config
```

Review at least the following:

- `DOMAIN` and `WEBSOCKET_ALLOWED_ORIGIN_PATTERNS`;
- `LETSENCRYPT_ENABLED` and the selected TLS files;
- `CONTAINER_TIMEZONE`;
- `REGISTRY_OUTBOUND_ALLOWED`, which defaults to `false`;
- `MYSCOUTEE_UPDATE_ENABLED`, which must remain `false` without a release public key;
- Firebase, which must remain `false` until an authenticated Operator has tested and activated its configuration.

The MongoDB and MinIO password placeholders have already been replaced with genuinely random values. Do not replace them merely with a manually selected “stronger” password; rotation requires a separate database/provider procedure that this manual does not implement.

6.4. Start for the First Time

```
sudo systemctl start myscoutee
sudo systemctl status myscoutee --no-pager
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh ps
```

6.5. Run Package-Provided Host Qualification

```
sudo /opt/myscoutee/packaging/scripts/qualify-deployment.sh \
  restart-and-check \
  --expected-version 1.4.1 \
  --timeout-seconds 300
```

This explicit command verifies the installed package version, restarts and waits for the systemd service, rejects restart loops, runs the application and MongoDB compatibility smoke, checks the Hungarian i18n bundle, and reports registry identity state. The MongoDB smoke creates, modifies, and drops the fixed temporary collection `_myscoutee_mongo_compat_smoke`; it does not operate on business collections. On failure, qualification prints scoped diagnostics and stops

`myscoutee.service` by default. Use `check` instead of `restart-and-check` only when the already-running service must not be restarted. The client installer already performs the corresponding mode and does not require this command to be repeated without cause.

6.6. Run the Independent External Smoke

After public DNS and a normally trusted TLS certificate are active, run the client bundle's read-only black-box smoke from a network outside the VM:

```
node smoke/deployment/run.mjs \
  --url https://app.example.com \
  --expected-version 1.4.1
```

Use Node.js 20 or later. The runner checks the normal TLS chain, HSTS, HTTP-to-HTTPS behavior, the forged-Host boundary, frontend/PWA assets, backend health/configuration/i18n/landing responses, and the anonymous Operator boundary. It deliberately has no insecure-TLS mode and refuses `NODE_TLS_REJECT_UNAUTHORIZED=0`. When local policy uses a private CA, provide it with `NODE_EXTRA_CA_CERTS=/approved/path/ca.pem`. A tunnel-based smoke is useful for transport isolation, but it does not replace a public-path test.

6.7. Record Commissioning Evidence

```
curl --fail --silent --show-error http://127.0.0.1/nginx-health
sudo docker exec myscoutee wget -qO- http://127.0.0.1:8080/health
curl --fail --silent --show-error https://app.example.com/api/health
```

Replace the domain with the actual public domain. The expected backend response is `{"status":"UP"}`. External HTTPS verification is not successful until a valid certificate is in place; using `-k` does not verify the TLS trust chain.

- [ ] The systemd unit is active.
- [ ] All six containers are running, and containers with health checks are healthy.
- [ ] The package-provided host qualification passed for version 1.4.1.
- [ ] The separate external smoke passed from an approved external network.
- [ ] HTTP application requests receive a 308 redirect to HTTPS.
- [ ] The public HTTPS certificate covers the correct domain.
- [ ] Operator sign-in succeeds.
- [ ] Deployment-specific Firebase and administrator configuration has been stored through the Operator interface or deliberately deferred.
- [ ] The first complete backup and off-host copy exist, or the limitation caused by an uninitialized registry has been documented.

7. Configuration and Secret Management

Path	Contents	Required protection
`/etc/myscoutee/myscoutee.env`	Non-secret production parameters, image references, gates, and paths	0644 at installation; modifications by root only

Path	Contents	Required protection
`/etc/myscoutee/myscoutee.secrets.env`	MongoDB/MinIO secrets, bootstrap identifier and password digest, and external provider placeholders	0600, root
`/etc/myscoutee/configuration/deployment-master-key`	At-rest encryption key for Operator-managed deployment secrets	numeric UID 0 / GID 10001, 0640; directory 0750
`/etc/myscoutee/registry/deployment-private-key.pem`	Deployment Ed25519 private signing key	numeric UID/GID 10001, 0600; directory 0700
`/etc/myscoutee/firebase-service-account.json`	Legacy file-backed startup credential; the current Operator-managed workflow leaves the package-created `{}` placeholder in place and stores encrypted credentials in MongoDB	numeric UID 0 / GID 10001, 0640; symbolic links are not permitted
`/etc/myscoutee/update/release-public-key.pem`	Ed25519 public verification key of the package publisher	numeric UID 0 / GID 10001, 0640

UID and GID `10001` are deliberate numeric container identities; a named host user or group is not required. Seeing `10001` in `ls -ln` output is expected. Do not create a matching host account or change these paths to root-only ownership. When an approved recovery procedure requires an ownership repair, use GNU `chown` with explicit `+`-prefixed numeric IDs, for example `chown +0:+10001 -- <path>`.

The Deployment Master Key and MongoDB Form One Recovery Unit

Losing or independently regenerating `deployment-master-key` makes the encrypted Operator configuration stored in MongoDB unrecoverable. Never replace the key as a “fix.” Back up and restore the key that belongs to the same database snapshot.

Controlled Configuration Change Procedure

1. Record the purpose, owner, and rollback plan for the change.
2. Create at least configuration and data backups in proportion to the risk.
3. Edit the appropriate file under `/etc/myscoutee`; do not place secrets in command-line arguments or shell history.
4. Validate Compose resolution:

```
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh config
```

5. Apply the change:

```
sudo systemctl restart myscoutee
```

6. Run health and functional checks, then close the change record.

Firebase, administrator, payment, email, social, and community settings managed through the authenticated Operator interface are deployment data. Saving Firebase configuration only stages it; live authentication and messaging require the real tests followed by the separate activation action.

8. Service Lifecycle

Start

```
sudo systemctl start myscoutee
```

Stop

```
sudo systemctl stop myscoutee
```

The systemd `ExecStop` runs Compose `down`. The containers and Compose network are removed, while the data directories and named volumes are retained.

Restart

```
sudo systemctl restart myscoutee
```

Automatic Startup

```
sudo systemctl enable myscoutee
sudo systemctl is-enabled myscoutee
```

Direct Compose Helper

```
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh ps
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh logs --tail=200
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh up -d
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh down
```

> ### An Active systemd Unit Does Not Mean Every Container Is Healthy
>
> `myscoutee.service` uses `Type=oneshot` and `RemainAfterExit=yes`. The `active (exited)` state proves that the Compose startup command ran, not that every container remains healthy. Always inspect both Compose state and container health.

Replica-Set Mode

MongoDB change-stream wakeup is optional. To enable it:

```
sudoedit /etc/myscoutee/myscoutee.env
# MONGO_REPLICA_SET_ENABLED=true
sudo systemctl restart myscoutee
```

The Compose helper then adds `docker-compose.replicaset.yml`, initializes a single `rs0` member, and retains a polling fallback. A backup and release-specific compatibility test are mandatory before switching modes.

9. Health and Operational Verification

9.1. Quick Status Check

```
sudo systemctl is-active myscoutee
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh ps
curl --fail --silent --show-error http://127.0.0.1/nginx-health
```

The Nginx health endpoint is exempt from the HTTP-to-HTTPS redirect and returns `UP`. It checks only the reverse-proxy process, not the complete application.

9.2. Component Checks

Component	Command	Expected result
Nginx	`sudo docker exec nginx nginx -t`	The configuration is syntactically valid and the selected key pair can be loaded.
Java API	`sudo docker exec myscoutee wget -qO- http://127.0.0.1:8080/health`	`{"status":"UP"}`; details are not public in production.
MongoDB	`sudo docker inspect --format '{{.State.Health.Status}}' mongodb`	`healthy`
Redis	`sudo docker exec redis redis-cli ping`	`PONG`
MinIO	`sudo docker inspect --format '{{.State.Health.Status}}' minio`	`healthy`, including seed readiness.
Explore registry	`sudo docker exec registry-demo /registry healthcheck`	Success with exit code 0.

9.3. Verify the Public Endpoint

```
curl --fail --silent --show-error https://app.example.com/api/health
curl --head http://app.example.com/
curl --head https://app.example.com/
```

Expected results: the first request returns an `UP` state; a normal HTTP request is redirected to HTTPS with status 308; the HTTPS response includes the one-year HSTS header.

9.4. MongoDB 8.0 Compatibility Smoke Test

```
sudo /opt/myscoutee/packaging/scripts/smoke-mongo-compatibility.sh
```

The smoke test verifies the configured and running MongoDB version, FCV, seed data, backend health, and public content queries.

> ### The Smoke Test Is Not Entirely Read-Only
>
> Using the application account, the script creates, modifies, queries, and then drops a temporary `_myscoutee_mongo_compat_smoke` collection. Treat it as a planned verification activity with appropriate authorization and change logging.

9.5. Version and Image Identity

```
dpkg-query -W -f='${Package} ${Version} ${Architecture}\n' myscoutee
sudo sed -n '1p' /opt/myscoutee/packaging/VERSION
sudo jq . /opt/myscoutee/packaging/IMAGE-MANIFEST.json
sudo docker inspect --format '{{.Name}} {{.Config.Image}} {{.Image}} {{if .State.Health}}
{{.State.Health.Status}}{{end}}' \
  nginx myscoutee minio registry-demo
```

The running image references and image IDs must match the installed manifest.

10. Logging, Monitoring, and Capacity

Service and Container Logs

```
sudo journalctl -u myscoutee.service --since "1 hour ago"
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh logs --since=1h --tail=500
sudo docker logs --since 1h myscoutee
sudo docker logs --since 1h nginx
sudo docker logs --since 1h mongodb
sudo docker logs --since 1h redis
sudo docker logs --since 1h minio
sudo docker logs --since 1h registry-demo
```

Privileged Worker Logs

```
sudo journalctl -u myscoutee-update.service --since "24 hours ago"
sudo journalctl -u myscoutee-tls.service --since "24 hours ago"
sudo journalctl -u myscoutee-tls-renew.service --since "30 days ago"
```

Additional persistent evidence for an update job:

```
/var/lib/myscoutee/update/runtime/jobs/<job-id>/status.json
/var/lib/myscoutee/update/runtime/jobs/<job-id>/events.ndjson
/var/lib/myscoutee/update/runtime/jobs/<job-id>/install.log
/var/log/myscoutee/update/<job-id>/request.json
```

TLS job status:

```
/var/lib/myscoutee/tls/runtime/jobs/<job-id>/status.json
/var/lib/myscoutee/tls/runtime/configuration.json
```

Capacity Checks

```
df -h /var/lib/myscoutee /var/cache/myscoutee-update /var/log/myscoutee /var/backups/myscoutee
sudo docker system df
sudo du -sh /var/lib/myscoutee/data/* /var/backups/myscoutee /var/cache/myscoutee-update 2>/dev/null
free -h
```

> ### No Packaged Log Rotation or Alerting System
>
> The Compose configuration does not set a logging driver or log-size limit, and the package includes no Prometheus scraper, alert manager, or disk alert. The operator must establish and test Docker daemon log rotation and host monitoring. In production, the API `/metrics` endpoint requires administrator privileges.

Minimum Alert Subjects

- any container in an `unhealthy`, `exited`, or restart-loop state;
- HTTP/HTTPS health failure or certificate expiration;
- capacity exhaustion in MongoDB/MinIO data, backup, update cache, or Docker storage;

- a failed or missing backup;
- `RECOVERY_REQUIRED`, a failed update, or a failed TLS job;
- registry connectivity, signature, or fingerprint errors;
- sustained growth in 5xx responses, memory use, CPU use, or restart count.

11. Backup

11.1. Routine Backup

```
sudo /opt/myscoutee/packaging/scripts/backup-production.sh
```

On an installed system, the default destination is `/var/backups/myscoutee/myscoutee-backup-YYYYMMDD-HHMMSS.tar.gz`. The archive mode is 0600.

11.2. Require a Complete Backup

```
sudo env MYSCOUTEE_BACKUP_REQUIRE_COMPLETE=true \
  /opt/myscoutee/packaging/scripts/backup-production.sh
```

The command does not produce a successful “complete” backup when any required component is missing.

11.3. Backed-Up Components

Component	Method	Notes
MongoDB application database	`mongodump --archive --gzip`	The running `mongodb` container is required.
Compose configuration	`.env`, `.env.secrets`, and `docker-compose-prod.yml`	Contains secrets.
Deployment master key	Exact file copy	Must be handled together with the MongoDB snapshot.
Real registry identity and state	Copies of the key directory and state/receipts	Only the complete key pair and state together are safe.
MinIO data	`tar.gz` of the host data directory	The script does not stop MinIO during copying.
Manifest	`backup-manifest.json`	Per-component complete/incomplete status.

11.4. Verify a Backup

```
sudo gzip -t /var/backups/myscoutee/myscoutee-backup-YYYYMMDD-HHMMSS.tar.gz
sudo tar -xOf /var/backups/myscoutee/myscoutee-backup-YYYYMMDD-HHMMSS.tar.gz \
  ./backup-manifest.json | jq .
sudo stat -c '%a %U:%G %s %n' \
```

```
  /var/backups/myscoutee/myscoutee-backup-YYYYMMDD-HHMMSS.tar.gz
sudo sha256sum /var/backups/myscoutee/myscoutee-backup-YYYYMMDD-HHMMSS.tar.gz
```

- [ ] The backup command returned exit code 0.
- [ ] The manifest `schemaVersion` is 1.
- [ ] `complete: true` is present when a complete backup is required.
- [ ] The archive mode is 0600.
- [ ] The SHA-256 digest was recorded separately in a protected register.
- [ ] An encrypted off-host copy exists.
- [ ] The retention and deletion policy has been applied.

The Backup Is Itself a Secret

The archive contains database credentials and, once it exists, the deployment private key. Mode 0600 on the host does not replace encryption, access logging, and integrity protection for off-host storage.

Verified Backup Limitations

- The script has no accompanying systemd timer or cron schedule.
- On an uninitialized deployment, the missing registry key/state makes the manifest incomplete; require-complete mode fails.
- The MinIO directory is not quiesced or stopped during backup; application consistency under write-intensive workloads must be tested separately.
- The backup excludes manual/Let’s Encrypt TLS material, the legacy file-backed Firebase credential, the update release public key, update/TLS job state, and the two Explore registry named volumes. Operator-managed Firebase credentials are encrypted MongoDB data and require the matching deployment master key.
- There is no built-in backup encryption, remote copy, rotation, or automated restore test.

12. Restore and Disaster Recovery

> **Destructive Operation**
>
> The restore performs `--drop` against MongoDB and, when MinIO restore is enabled, deletes the current MinIO data directory. Before proceeding, exclude user traffic, preserve the pre-restore state, and verify that the correct backup has been selected.

12.1. Preconditions

- [ ] The selected archive's SHA-256 digest and manifest have been verified.
- [ ] The target is the same logical deployment or part of a documented clean-host disaster-recovery drill.
- [ ] Compatibility between the installed package and the backup has been verified for the specific release.
- [ ] The `mongodb` container is running.
- [ ] User traffic is isolated.
- [ ] A separate backup of the pre-restore state has been created, if still possible.
- [ ] The required `RESTORE_MINIO`, `RESTORE_CONFIG`, and `RESTORE_REGISTRY` decisions have been approved.

12.2. Basic Restore — MongoDB and Its Associated Master Key

```
sudo env CONFIRM_RESTORE=true \
  /opt/myscoutee/packaging/scripts/restore-production.sh \
  /var/backups/myscoutee/myscoutee-backup-YYYYMMDD-HHMMSS.tar.gz
```

The script requires both the MongoDB data and the deployment master key. It stages the key, stops the Java API, runs `mongorestore`, and activates the key only after the restore succeeds.

12.3. Extended Restore

```
sudo env CONFIRM_RESTORE=true \
  RESTORE_MINIO=true \
  RESTORE_CONFIG=true \
  RESTORE_REGISTRY=true \
  /opt/myscoutee/packaging/scripts/restore-production.sh \
  /var/backups/myscoutee/myscoutee-backup-YYYYMMDD-HHMMSS.tar.gz
```

Switch	Effect	Risk
`RESTORE_CONFIG=true`	Copies the saved `.env`, `.env.secrets`, and Compose file back into the server tree.	In an installed environment, verify the relationship between `/etc/myscoutee` and the server symbolic links after restore.
`RESTORE_MINIO=true`	Stops MinIO, deletes the current data directory, extracts the backup, and restarts MinIO.	The current object state is lost.

Switch	Effect	Risk
`RESTORE_REGISTRY=true`	Validates and restores the complete deployment key pair, state, and receipt history.	Transfers signing control of the registered deployment; never clone it to an independent installation.

Registry restore rejects a key-only backup, a mismatched Ed25519 key pair, an invalid fingerprint/state binding, and invalid receipt cryptography.

12.4. Post-Restore Verification

```
sudo systemctl restart myscoutee
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh ps
curl --fail --silent --show-error http://127.0.0.1/nginx-health
sudo docker exec myscoutee wget -qO- http://127.0.0.1:8080/health
sudo /opt/myscoutee/packaging/scripts/smoke-mongo-compatibility.sh
```

- [ ] MongoDB and the deployment master key come from the same recovery set.
- [ ] The backend, Nginx, MongoDB, Redis, MinIO, and Explore registry are healthy.
- [ ] Operator sign-in and deployment-specific provider configuration work.
- [ ] After a registry restore, the deployment fingerprint, state, and receipt history match.
- [ ] After a MinIO restore, sampled object access succeeds.
- [ ] External registry outbound access was re-enabled only after verification.

> **Clean-Host Disaster Recovery Is Not Generally Proven**
>
> The restore script includes strong technical checks, but the repository qualification document requires a separate clean-host restore and an upgrade drill from the oldest supported version for every release. Source tests or the successful extraction of one archive alone do not prove RTO, RPO, or complete recoverability.

> **Incomplete Cryptographic Restore**
>
> If replacement of the MongoDB/key pair or registry identity/state is interrupted, the script may intentionally leave the Java API stopped. Do not restart the API blindly; first establish the cause and verify the key/data pair in use.

13. TLS and Certificate Lifecycle

Production Nginx always expects TLS. It redirects normal HTTP application requests to HTTPS with status 308, except for `/nginx-health` and the ACME challenge path. HTTPS responses include a one-year HSTS header.

13.1. TLS Modes

Mode	Switch	Container path	Host storage
Operator-managed	`LETSENCRYPT_ENABLED=false`	`/etc/nginx/keys/nginx.crt` and `nginx-nopass.key`, or a configured absolute path	`/opt/myscoutee/server/docker/conf/nginx/keys`
Let’s Encrypt	`LETSENCRYPT_ENABLED=true`	`/etc/letsencrypt/live/<DOMAIN>/fullchain.pem` and `privkey.pem`	`/var/lib/myscoutee/data/certbot/conf`

When both operator-managed TLS files are absent, package configuration creates a matching self-signed RSA-2048/SHA-256 bootstrap pair valid for 3650 days, with common name `localhost` and subject alternative names `DNS:localhost` and `IP:127.0.0.1`. The certificate is mode 0644 and its private key is mode 0600. This pair is for local inspection only and is not a trusted public certificate for the deployment FQDN. An existing complete pair is preserved byte-for-byte; if exactly one file exists, it is preserved and configuration fails closed, as described in the TLS backup-gap callout below.

13.2. Recommended Operator Workflow

1. Point public DNS to the deployment and make TCP ports 80 and 443 reachable.
2. Sign in to Configuration as an authenticated Operator.
3. Enable HTTPS management and choose automatic or manual mode.
4. Enter the public domain and, in automatic mode, the certificate contact email address.
5. Run the domain and certificate test.
6. Save the block; changing the toggle alone creates only a draft.
7. In automatic mode, enable and save automatic renewal.
8. Verify that Nginx restarts and that the external trust chain is valid.

The privileged TLS worker accepts only a fixed-schema JSON request; it does not accept arbitrary commands, paths, images, or URLs from the browser. For a manual key pair, it verifies that the certificate and key match, that the certificate covers the domain, and that the certificate remains valid for at least 24 hours.

13.3. CLI Recovery for the First Let’s Encrypt Certificate

```
sudo systemctl stop myscoutee
sudo install -d -m 0755 /var/lib/myscoutee/data/certbot/conf
sudo docker run --rm --name myscoutee-certbot-bootstrap \
  -p 80:80 \
  -v /var/lib/myscoutee/data/certbot/conf:/etc/letsencrypt \
  certbot/certbot:v5.4.0 certonly --standalone \
  --non-interactive --agree-tos --no-eff-email \
  --email operator@example.com -d app.example.com
sudo sed -i \
  -e 's/^DOMAIN=.*/DOMAIN=app.example.com/' \
  -e 's/^LETSENCRYPT_ENABLED=.*/LETSENCRYPT_ENABLED=true/' \
  /etc/myscoutee/myscoutee.env
sudo systemctl start myscoutee
```

13.4. Manual Renewal During Recovery

```
sudo systemctl stop myscoutee
sudo docker run --rm --name myscoutee-certbot-renew \
  -p 80:80 \
  -v /var/lib/myscoutee/data/certbot/conf:/etc/letsencrypt \
  certbot/certbot:v5.4.0 renew --standalone
sudo systemctl start myscoutee
```

13.5. Verify Automatic Renewal

```
sudo systemctl status myscoutee-tls-renew.timer --no-pager
sudo systemctl list-timers myscoutee-tls-renew.timer
sudo journalctl -u myscoutee-tls-renew.service --since "30 days ago"
```

The timer runs daily in persistent mode, with a randomized delay of up to two hours. It renews only when saved `AUTOMATIC` mode is enabled and `autoRenew=true`.

13.6. Verify the Certificate Externally

```
openssl s_client -connect app.example.com:443 -servername app.example.com \
  </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer -dates
curl --fail --silent --show-error https://app.example.com/api/health
```

TLS Backup Gap

The production backup script saves neither the manual Nginx key pair nor the Let’s Encrypt directory. These require a separate encrypted backup and reissuance procedure. Normal remove, deconfigure, upgrade, and same-version reinstall preserve a complete manual Nginx certificate/key pair. Package configuration creates the bootstrap pair only when both files are absent; if exactly one exists, it preserves that file and fails closed so the Operator can restore its counterpart. An explicit `apt purge` deletes the pair with `/opt/myscoutee`.

Enabling and Provisioning Are Separate Operations

`LETSENCRYPT_ENABLED=true` does not request a certificate; it only activates an existing certificate/key pair located exactly under `DOMAIN`. Nginx startup fails when the pair is missing or invalid.

14. Operator-Approved Updates

A registry `UPDATE` announcement only reports an available release. Download and installation occur solely after explicit approval by an authenticated Operator. The Debian package installs this local update mechanism; no remote shell session or separate remote automation is required, and neither the package nor the worker performs unattended polling and installation.

Signed 1.4.1 release and retained data

Obtain the Debian installer, client tools, checksums and signed release metadata from MyScoutee 1.4.1. Verify the pinned release key and artifact identities using the existing client installer. The signed update catalog's minimum installed package is 1.4.0; the separately verified Netcup 1.3.0-to-1.4.1 operation used the signed manual client installer.

Before upgrading, require a complete backup and retain the deployment secrets, configuration master key, registry identity, TLS material and operator media using their documented backup owners. Package migrations run in order through 019. The Feedback migration retains historical answers and stops on conflicting canonical data; permission migration preserves explicit operator restrictions and grants.

After installation, run host qualification and the external smoke with expected version 1.4.1. Verify the rendered frontend, all nine service health states, retained records and media, provider settings and existing TLS/secret identities. A healthy container alone does not establish a working browser session.

The 1.4.1 Netcup upgrade passed these recorded checks. Rollback, power-loss, clean-host restore and sustained-load drills remain separate. Exact package hashes, image digests and qualification limits are in the release notes.

14.1. Prepare the Host

1. Through an independent channel, verify the full SPKI DER SHA-256 fingerprint of the vendor's Ed25519 release public key.
2. Install the key:

```
sudo install -m 0640 -- release-public-key.pem \
  /etc/myscoutee/update/release-public-key.pem
sudo chown +0:+10001 -- \
  /etc/myscoutee/update/release-public-key.pem
sudo chmod 0640 -- \
  /etc/myscoutee/update/release-public-key.pem
```

3. Verify the local fingerprint:

```
sudo openssl pkey -pubin \
  -in /etc/myscoutee/update/release-public-key.pem \
  -outform DER | sha256sum
```

4. Configure the gate:

```
sudoedit /etc/myscoutee/myscoutee.env
# MYSCOUTEE_UPDATE_ENABLED=true
sudo systemctl restart myscoutee
```

```
sudo systemctl restart myscoutee-update.path
```

5. Verify the path unit:

```
sudo systemctl status myscoutee-update.path --no-pager
```

14.2. Operator Approval

Before selecting **Update deployment**, open **Operator → Community**, locate the current published `UPDATE` announcement, and expand **Details**. Proceed only when:

- ☐ the announcement is **Published**, not archived or withdrawn;
- ☐ the displayed target version is the approved release and the separately approved release evidence confirms its release channel;
- ☐ the artifact size and compatibility range are acceptable for the installed version and maintenance window;
- ☐ **SHA-256** is present in canonical `sha256:<64-lowercase-hex-characters>` form;
- ☐ **Package signing key ID** is the expected `pkey_<32-lowercase-hex-characters>` value derived from the independently pinned release key;
- ☐ **Canonical Ed25519 signature** is present, and **Signature verification** explicitly reads **Release metadata verified by the pinned key**; never proceed when it reads **Release metadata not verified by the pinned key**;
- ☐ the artifact address is the expected credential-free HTTPS `.deb` URL and is labelled **Verified release URL**;
- ☐ free space for staging and cache and the ability to create a complete pre-update backup have been confirmed.

Then open **Operator → Updates** and confirm that **Available version** exactly matches the version reviewed in Community. Selecting **Update deployment** is the approval action and begins the update immediately; the current Updates popup does not repeat the cryptographic fields or add another confirmation dialog.

What the Pre-Approval Verification Proves

The shortened package-signing key ID does not replace the independently verified full SPKI fingerprint in 14.1. The positive UI status authenticates the signed version, channel, size, and digest metadata with the locally pinned key; signature presence alone is not proof. The package bytes have not yet been downloaded at this point. Their final size and SHA-256 are checked during the subsequent verification phase and again by the privileged host worker.

14.3. Execution Chain

1. ☐ The Java API starts from the credential-free, fragment-free HTTPS `.deb` URL in the registry-signed manifest. It may follow at most five loop-free HTTPS redirects that remain credential- and fragment-free, accepts only a final HTTP 200 response, enforces the signed and configured size limits while streaming, and verifies final size and SHA-256.
2. ☐ The API writes to staging, then publishes a fixed-schema request to the host inbox.
3. ☐ The systemd path unit starts a root worker, which atomically claims or quarantines the input.

4. ☐ The worker independently revalidates the package name, version, architecture, size, digest, release key ID, Ed25519 signature, image manifest, and free space.
5. ☐ It creates a mandatory production backup with `complete: true`.
6. ☐ It runs `apt-get install` from a local file.
7. ☐ It restarts the stack and, for up to 300 seconds, verifies the exact reference, ID, and health of each of the four managed running images.

Default storage limits:

- ☐ Java staging safety reserve: 512 MiB in addition to the artifact size;
- ☐ root cache minimum free reserve: 512 MiB in addition to the artifact size;
- ☐ root cache retention: two jobs;
- ☐ maximum artifact size: 4 GiB.

14.4. Monitor Progress

```
sudo journalctl -fu myscoutee-update.service
sudo cat /var/lib/myscoutee/update/runtime/jobs/<job-id>/status.json
sudo tail -f /var/lib/myscoutee/update/runtime/jobs/<job-id>/install.log
```

Terminal states are `COMPLETED`, `FAILED`, `REJECTED`, and `RECOVERY_REQUIRED`.

No Automatic Data Rollback

If installation is partial or cannot be verified, the worker sets `RECOVERY_REQUIRED`. It does not automatically restore the previous data or package. Use the backup recorded in the status, the installation log, and the release-specific recovery procedure.

14.5. Recovery

```
sudo journalctl -u myscoutee-update.service
sudo cat /var/lib/myscoutee/update/runtime/jobs/<job-id>/status.json
sudo cat /var/lib/myscoutee/update/runtime/jobs/<job-id>/install.log
sudo env CONFIRM_RESTORE=true \
  /opt/myscoutee/packaging/scripts/restore-production.sh \
  /var/backups/myscoutee/<backup-recorded-in-status>.tar.gz
```

Uninitialized Registry and the Update Gate

The backup helper treats the registry key and state as part of completeness. Before registry initialization, a backup with `MYSCOUTEE_BACKUP_REQUIRE_COMPLETE=true` fails, so the privileged update worker does not reach installation. Do not bypass this by manually creating files.

During a failed Debian package transaction, `postinst abort-upgrade`, `abort-remove`, and `abort-deconfigure` return without rerunning the full configuration path. This protects existing keys, bootstrap material, TLS files, and configuration from recovery side effects; it is not an automatic

rollback or repair. Inspect the APT, dpkg, and service logs, recover the package transaction through the approved package-manager procedure, and then rerun the explicit deployment qualification before restoring traffic.

Manual `apt install ./new-package.deb` is technically possible, but it bypasses the Operator update rail's manifest, signature, complete-backup, and progress controls. Use it for a production update only as an approved, release-specific documented exception.

15. Registry Integration and Deployment Identity

15.1. Initial State

Package installation does not select a registry, create a deployment identity, register, or send telemetry. A new node is in the *installed/uninitialized* state.

Production defaults:

```
REGISTRY_ENABLED=false
REGISTRY_OUTBOUND_ALLOWED=false
REGISTRY_OPERATOR_REGISTRATION_CONFIRMED=false
REGISTRY_BASE_URL=
REGISTRY_SCOPE=
REGISTRY_CONNECT_TIMEOUT=5s
REGISTRY_REQUEST_TIMEOUT=15s
REGISTRY_RETRY_DELAY=15m
REGISTRY_MAX_RESPONSE_BYTES=1048576
REGISTRY_EXPECTED_KEY_FINGERPRINT=
REGISTRY_TRUST_ON_FIRST_USE=false
REGISTRY_ALLOW_INSECURE_HTTP=false
REGISTRY_INSECURE_HTTP_ALLOWED_HOSTS=
```

15.2. Registration Flow in the Current Registry Dialog

1. The system operator sets the host-level `REGISTRY_OUTBOUND_ALLOWED` kill switch to `true` only after outbound registry traffic has been approved.
2. The authenticated Operator opens the Registry dialog. The current field is a read-only catalog selector: the Operator selects one of the offered HTTPS endpoints and its cataloged scope.
3. The Operator initiates **Register**. The frontend sends the selected `registryBaseUrl` and `expectedRegistryScope` values to `/api/operator/registry/register`.
4. As part of the same registration action, the Java backend performs a live registry identity inspection, verifies the expected scope and any compatible existing binding, and persists the selection.
5. The backend locally creates or loads the Ed25519 deployment identity and performs registration synchronization.
6. The backend verifies the registry-signed receipt against the pinned registry public key and stores it persistently.
7. The dialog displays the lifecycle state, current URL, and success or failure of the action. Use **Disconnect** to terminate an active connection.

TOFU and Plain HTTP

Trust on first use and insecure HTTP are disabled by default in production. Do not enable them for a real registry merely for convenience. An insecure HTTP exception is only for controlled development hosts with an explicit allowlist; wildcards are not supported.

Control Gap: the Planned Two-Phase Confirmation Is Not Connected to the Current Dialog

The project context specifies a separate read-only inspection, display of the registry signing identity and fingerprint, and then explicit Operator confirmation. The backend `/inspect` and `/confirm` APIs and corresponding frontend store methods exist, but the current Registry dialog does not call them: **Register** invokes the single-phase `/register` action directly and does not display a separate fingerprint confirmation. Therefore, the current dialog must not be documented as satisfying the planned two-phase human identity control. Before enrollment with a real production registry, the release manager must treat this as an open control gap, independently approve the catalog endpoint and scope, and document either the accepted residual risk or the corrected release.

15.3. Persistent Identity and State

```
/etc/myscoutee/registry/deployment-private-key.pem
/etc/myscoutee/registry/deployment-public-key.der
/var/lib/myscoutee/registry/deployment-state.json
/var/lib/myscoutee/registry/receipts/
```

The private key is PKCS#8 PEM with mode 0600; the public key is X.509 SPKI DER. The backend runs as unprivileged UID/GID 10001 but receives writable access to the registry key directory because the explicit Operator action publishes the first key pair there.

Boundary of Backend Compromise

The Java backend is the registry's only network client and possesses the deployment private key. Compromise of the backend process also endangers this key; never share the directory with another service.

15.4. Changing Registries

Changing to another registry is not a simple URL edit. The current registry must first receive a signed unregister/deactivation; only then may the old binding and receipts be archived while retaining the node key. History, claims, ledger entries, and registry identity are not migrated automatically.

15.5. Registry Operations Checklist

- [ ] The state of the host outbound kill switch is documented.

- [ ] The registry URL and scope selected from the read-only catalog have prior release- or deployment-level approval.
- [ ] The risk decision for the gap between the current single-phase dialog and the planned two-phase fingerprint confirmation is documented.
- [ ] The registry receipt verified by the backend and the deployment state are preserved in backups.
- [ ] The deployment key and its associated state/receipts are included together in the backup.
- [ ] After restore or a registry change, outbound access was enabled only with verified state.
- [ ] The internal Explore registry is not confused with the real registry.

16. Security Operations

Host and Network

- Expose only ports 80 and 443 publicly; the MongoDB host mapping must remain on loopback.
- MinIO, Redis, the Java API, and the Explore registry publish no host ports.
- Treat access to the Docker socket and membership in the `docker` group as equivalent to root privileges.
- Use a separate host-patching process for operating-system and Docker security updates; the MyScoutee package does not perform aggressive system upgrades.
- Verify that administrative access will be preserved before changing firewall rules.

Application and Container Boundaries

- The Java production image runs as UID/GID 10001 and exits when an out-of-memory condition occurs.
- The Explore registry uses a read-only root filesystem, `no-new-privileges`, and drops all capabilities.
- The TLS and update workers run as root but use fixed host paths and narrowly defined input schemas.
- For managed application images, `pull_policy: never` prevents silent image replacement behind a tag.
- The Nginx and backend upload limit is 10 MB.
- The backend does not expose detailed production health information; metrics require administrator privileges.

Secret and Key Management Rules

- [ ] Never place a private key or service account in Git, an image, a ticket, or chat.
- [ ] The release private key must never exist on an application deployment or registry server.
- [ ] Backups are encrypted, access-logged, and stored off-host.
- [ ] The deployment master key cannot be rotated without atomically re-encrypting every encrypted record.
- [ ] The registry deployment key may be restored only with a complete state/receipt recovery set.
- [ ] The shown-once Operator credential and installation log are protected.
- [ ] External provider credentials are handled through the authenticated deployment-configuration process; the package contains only empty templates.

Separate Public and Secret Data

Endpoints, fingerprints, public verification keys, Firebase web-client identifiers, and the VAPID public key are not server-authentication secrets. The Firebase service-account private key, payment/email/social tokens, MongoDB and MinIO passwords, deployment private key, and master key are secrets.

17. Troubleshooting and Incident Response

17.1. The First Five Minutes

1. Record the time, symptoms, affected users, and most recent change.
2. Do not delete containers, volumes, keys, update state, or logs.
3. Check systemd and Compose state:

```
sudo systemctl status myscoutee --no-pager
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh ps
```

4. Capture scoped status and recent journal evidence:

```
sudo /opt/myscoutee/packaging/scripts/deployment-diagnostics.sh
```

5. Inspect the relevant container and worker logs.
6. Check free disk space, memory, and Docker storage.
7. For cryptographic or data-consistency errors, isolate traffic and stop further automated actions.

17.2. Symptom-Based Troubleshooting

Symptom	Check	Safe next step
`myscoutee.service` is active, but the site is unavailable	Compose `ps`, Nginx/backend health, container logs, listeners on 80/443, and the firewall.	Correct the cause of the unhealthy component; do not infer health from systemd's active state.
Nginx restart loop	`docker logs nginx`, `DOMAIN`, existence of the selected certificate/key, and `docker exec nginx nginx -t` if the container is running.	Restore the last known-good TLS pair and environment settings.
Backend unhealthy	MongoDB, Redis, and MinIO health; backend log; master-key path and permissions; Firebase file if enabled.	Restore the primary dependency; do not regenerate the master key.

Symptom	Check	Safe next step
MongoDB unhealthy or authentication error	MongoDB log, disk, `.env.secrets`, initialization data directory, and replica-set state.	Do not merely change the password in the environment file; database-user rotation requires a separate procedure.
MinIO unhealthy	MinIO log, data directory, permissions, free space, and seed-readiness health.	Back up the directory before deleting data; restore only from an approved archive.
Registry unavailable	Outbound kill switch, HTTPS URL/DNS/TLS, fingerprint, timeout, registry state, and backend log.	Do not enable TOFU or plain HTTP; the local application must remain available during a temporary registry failure.
Update `FAILED`	Job status, events, installation log, journal, release key, digest, disk, and backup manifest.	Correct the verified cause; do not recreate the same job ID.
Update `RECOVERY_REQUIRED`	Installed dpkg version, image manifest, running image IDs, and the backup recorded in status.	Use release-specific recovery/restore instead of automatic retry.
TLS renewal failed	DNS, public port 80, ACME challenge, timer/service journal, Certbot image, and certificate path.	Correct reachability; use the documented manual recovery command if necessary.
Disk full	`df`, Docker `system df`, backups, MongoDB/MinIO, update cache, and log size.	Delete only on the basis of evidence and the retention policy; do not run a blind `docker system prune`.

17.3. Evidence Package

For an incident, retain at least the following with timestamps and share it without secrets:

- host/distribution/kernel and Docker and Compose versions;
- installed dpkg version, image manifest, and running image IDs;
- systemd and relevant container status and logs;
- disk and memory state;
- update/TLS job status, events, and installation log;
- backup manifest and SHA-256, without the archive contents;
- the most recent configuration, TLS, registry, or release change.

Prohibited Troubleshooting “Fixes”

Do not delete or regenerate the deployment master key, registry private key/state, MongoDB data directory, MinIO data, update runtime evidence, or Docker volumes. Do not enable registry TOFU/plain HTTP, run blind purge/prune commands, or restart the API during an incomplete key/data restore.

18. Maintenance Checklists

The frequencies below are operational baselines, not built-in automation. Local SLA, RPO, and workload may require a stricter schedule.

Daily

- [ ] Check systemd and the status of all six containers.
- [ ] Check Nginx and backend health.
- [ ] Review new 5xx responses, restart loops, and update/TLS/registry errors.
- [ ] Check disk and memory thresholds.
- [ ] Verify the expected backup job's success and manifest.

Weekly

- [ ] Review growth in Docker and application logs.
- [ ] Verify the backup's off-host copy and SHA-256 digest.
- [ ] Check certificate expiration and renewal-timer state.
- [ ] Review the update cache and failed/rejected inbox and archive entries.
- [ ] Review MongoDB/MinIO storage trends and Docker image storage.

Monthly or Before a Change

- [ ] Perform a restore test in an isolated, non-production environment.
- [ ] Review Operator, administrator, root, and Docker access.
- [ ] Verify the validity of external provider credentials and connections.
- [ ] Review the registry endpoint/fingerprint and the ability to back up receipts.
- [ ] Review host OS/Docker patching, vulnerabilities, and capacity.

Before Every Release

- [ ] The candidate `.deb`, every image ID/digest, and the source commit are bound to one evidence package.
- [ ] Artifact preflight and all packaging/protocol tests have passed.
- [ ] The upgrade and interruption drill from the oldest supported version is complete.
- [ ] Clean-host restore, multi-registry, load, and security drills are complete.
- [ ] No qualification row or unaccepted security finding remains open.

```
MYSCOUTEE_QUALIFICATION_EVIDENCE_DIR=/absolute/evidence/1.2.0 \
  bash packaging/release/qualify-release-artifacts.sh \
  /absolute/myscoutee_1.2.0_amd64.deb \
  /absolute/myscoutee_1.4.1_amd64.deb \
  myscoutee-registry:1.2.0-prod
```

This is a read-only artifact preflight; it does not replace host-modifying upgrade and restore drills.

19. Removal and Reinstallation

Remove the Package While Retaining Data

```
sudo apt remove myscoutee
```

The process stops and removes the Compose containers and disables the application, update, and TLS units. It retains substantial state under `/etc/myscoutee` and `/var/lib/myscoutee`, the complete manual TLS pair under the otherwise removed package tree, the demo registry volumes, and MyScoutee image tags for a later reinstallation. The package-owned archive at `/usr/share/myscoutee/myscoutee-client-tools_1.4.1.tar.gz` is removed; a separately downloaded and extracted workstation copy is unaffected.

Manual TLS Pair Is Preserved Until Purge

Normal remove, deconfigure, upgrade, and reinstall preserve a complete `nginx.crt` / `nginx-nopass.key` pair. This lifecycle preservation is not a backup: keep a separate protected copy or reissuance procedure. A partial pair makes package configuration fail closed, and purge removes the entire `/opt/myscoutee` tree.

Complete Purge

```
sudo apt purge myscoutee
```

The trusted workstation bundle also provides `./purge.sh`, which invokes remote `apt-get purge -y myscoutee` and deletes the uploaded `.deb`. It has no interactive confirmation or dry-run mode. Use it only after completing the checklist below; it is not part of a normal reinstall or upgrade.

Purge Causes Irreversible Data Loss

Purge deletes the `/etc/myscoutee`, `/opt/myscoutee`, `/var/lib/myscoutee`, `/var/log/myscoutee`, and update-cache trees; both Explore registry volumes; and every `myscoutee-api`, `myscoutee-nginx`, `myscoutee-minio`, and `myscoutee-registry` image tag. It does not delete shared upstream MongoDB/Redis images or `/var/backups/myscoutee`. A complete off-host backup and registry identity/state verification are mandatory before purge.

Pre-Removal Checklist

- [ ] Final removal has been approved.
- [ ] A complete backup, its SHA-256 digest, and an off-host copy are available.
- [ ] The separate backup gaps for TLS, Firebase, and the update key have been addressed.
- [ ] The need for a registry deactivation/exit procedure has been assessed.
- [ ] The legal basis for retaining the deployment private key and receipt history is documented.
- [ ] After purge, the backup remains reachable and readable.

20. Known Operational Limitations

Area	Limitation	Required operational treatment
Sizing	No qualified universal throughput, user-count, or capacity limit, and no container resource limits.	Use the Chapter 4 planning baseline, then perform release- and deployment-specific load/capacity testing and host monitoring.
High availability	Single-host Compose and, optionally, one MongoDB replica-set member; this is not host-level HA.	Define the expected SLA accordingly; external HA requires separate design.
Redis	No declared persistent volume.	Operation must tolerate cache loss, and restart behavior must be tested.
Backup automation	No packaged timer, rotation, remote destination, or encryption.	External scheduler, retention, encryption, monitoring, and restore testing.
Backup completeness	TLS, the legacy file-backed Firebase credential, update key/state, and Explore volumes are absent from the production backup.	A separate secret/key backup or reissuance procedure.
MinIO backup	A filesystem tar is created while the service runs, without quiescing it.	Consistency testing under write load and, if needed, a maintenance window.
Logs and alerts	No Compose log limit, scraper, or alert manager.	Host/Docker log rotation and external monitoring.
Updates	No automatic rollback; without a complete registry backup, the gate cannot be satisfied.	A qualified recovery drill and Operator-controlled restore.
Clean-host restore	It cannot be considered generally proven by source tests.	An actual clean-host/VM drill for every release candidate.
Image acquisition	The standard package contains five first-party images, but it must pull MongoDB/Redis if missing; Certbot is required for TLS operations.	Preverified outbound network access or an explicit offline image bundle for the release.

Area	Limitation	Required operational treatment
Restricted-network integrations	Country-local hosting does not make Firebase or another configured external provider local.	Qualify each provider from the actual network, keep it disabled when unavailable, or use a separately supported regional integration.
Registry topology	One deployment selects one real registry endpoint and scope; there is no active-active selection, cross-scope replication, or automatic failover.	Use separate regional deployment populations and follow the signed deactivation/change procedure before moving an existing deployment.
Source and package	The two production Compose variants are not identical.	On an installed system, use only the `/opt/myscoutee` wrapper and packaged Compose file.

Principle for Managing Limitations

The limitations listed here do not authorize bypassing controls. The appropriate response is a documented compensating control, a qualification drill, or a new software release.

21. Command and Path Quick Reference

Common Commands

Purpose	Command
Start	`sudo systemctl start myscoutee`
Stop	`sudo systemctl stop myscoutee`
Restart	`sudo systemctl restart myscoutee`
Stack status	`sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh ps`
Stack logs	`sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh logs --tail=200`
Host qualification	`sudo /opt/myscoutee/packaging/scripts/qualify-deployment.sh check --expected-version 1.4.1 --timeout-seconds 300`
Scoped diagnostics	`sudo /opt/myscoutee/packaging/scripts/deployment-diagnostics.sh`
External smoke (client)	`node smoke/deployment/run.mjs --url https://app.example.com --expected-version 1.4.1`
Nginx health	`curl -fsS http://127.0.0.1/nginx-health`

Purpose	Command
Backend health	`sudo docker exec myscoutee wget -qO- http://127.0.0.1:8080/health`
Complete backup	`sudo env MYSCOUTEE_BACKUP_REQUIRE_COMPLETE=true /opt/myscoutee/packaging/scripts/backup-production.sh`
Guarded restore	`sudo env CONFIRM_RESTORE=true /opt/myscoutee/packaging/scripts/restore-production.sh <backup.tar.gz>`
MongoDB smoke test	`sudo /opt/myscoutee/packaging/scripts/smoke-mongo-compatibility.sh`
Update log	`sudo journalctl -u myscoutee-update.service`
TLS renewal log	`sudo journalctl -u myscoutee-tls-renew.service`

Important Host Paths

Path	Purpose
`/opt/myscoutee`	Packaged application, Compose files, scripts, and image manifest.
`/usr/share/myscoutee/myscoutee-client-tools_1.4.1.tar.gz`	Package-owned version-matched client archive for controlled SFTP retrieval; never extract or execute it on the host.
`/etc/myscoutee`	Deployment configuration, secrets, and keys.
`/var/lib/myscoutee/data`	MongoDB, MinIO, and Certbot data.
`/var/lib/myscoutee/registry`	Real registry state and receipts.
`/var/lib/myscoutee/update`	Update inbox, archive, and runtime state.
`/var/lib/myscoutee/tls`	TLS inbox, archive, configuration, and job status.
`/var/cache/myscoutee-update`	Root-owned update artifact cache.
`/var/log/myscoutee/update`	Audit evidence for accepted update requests.
`/var/backups/myscoutee`	Default production backup destination; retained after purge.

Emergency Sequence

1. Isolate traffic.
2. Preserve state and evidence.
3. Verify key/data pairing.
4. Perform release-specific recovery or a guarded restore.
5. Complete health, identity, and functional verification.
6. Restore traffic and registry outbound access in a controlled manner.

7. Close the incident and review the manual and procedures.

Appendix A — Source Build, Package Build, and Development Profile

This appendix covers the repository's complete lifecycle. The build and development profiles described here are not the same as the `apt`-installed production profile operated in the main body of this manual.

A.1. Build Production Artifacts from Source

From the repository root:

```
MYSCOUTEE_VERSION=1.4.1 \
  bash packaging/release/build-production.sh
```

By default, the script builds and synchronizes the production frontend bundle, runs the Gradle `bootJar` build, creates the base Nginx image, and then builds the source-production Compose images. The build environment must also have access to the adjacent MyScoutee registry source.

A.2. Build the Debian Package and Client Bundle

In one step, including an automatic artifact build:

```
MYSCOUTEE_VERSION=1.4.1 \
  bash packaging/release/package-production.sh
```

If the preceding production build was created from exactly the same source state and version, avoid rebuilding it:

```
MYSCOUTEE_VERSION=1.4.1 \
  BUILD_ARTIFACTS=false \
  bash packaging/release/package-production.sh
```

Expected outputs are `packaging/dist/myscoutee_1.4.1_amd64.deb` and `packaging/dist/myscoutee-client-tools_1.4.1.tar.gz`. The package build saves the five managed MyScoutee images into the Debian archive, creates an image manifest, runs a credential audit, embeds a byte-identical copy of the versioned client archive at `/usr/share/myscoutee/myscoutee-client-tools_1.4.1.tar.gz`, and emits the standalone workstation archive. It does not install expanded client scripts on the deployment host. The source-only convenience entry point `packaging/release/build-debian-release.sh` invokes the same package builder.

Building does not by itself establish a published release. On an isolated release workstation, use the source-only `packaging/release/sign-release-artifacts.sh` with an approved pre-existing Ed25519 PKCS#8 private key to validate and sign the matching Debian and client artifacts and to produce `myscoutee-release_1.4.1.json`. The signing tool neither creates nor copies the private key. Publish the Debian package, client archive, signed manifest, and public key; distribute the full SPKI fingerprint through an independent trusted channel. Production use still requires release qualification and approval.

The release lifecycle is therefore: build both artifacts; sign them on the controlled release host; publish the two artifacts, signed manifest, and public key; authenticate the full SPKI fingerprint independently; verify the client archive with an already trusted verifier; extract it; and let its

installer reverify the Debian package before SCP and APT.

A.3. Development Stack

```
cd server
docker compose up --build
```

The development stack uses hot reload, workspace mounts, development/demo helpers, and services that can be inspected on loopback. The sibling `myscoutee-registry` repository is required to build the real Go registry sandbox.

> ### The Profiles Are Not Interchangeable
>
> The development stack, source `server/docker-compose-prod.yml`, and packaged `/opt/myscoutee/server/docker-compose-prod.yml` use different build policies, host paths, health timing, and security/operations boundaries. Development `docker compose up --build` does not replace Debian package installation, the systemd lifecycle, production TLS, guarded backup/restore, or the signed update rail.

End of document — MSC-OM-001 / 1.4.1 / Released

Applicable software version: MyScoutee 1.4.1 · Release date: 2026-10-17 · Language: EN